

Computable functions and predicates

- Computable functions and predicates are foundational elements in artificial intelligence and logic-based systems, used for modeling how AI systems process information and make decisions. They allow AI systems to reason about data and express relationships between entities.
- All the simple facts can be expressed as combination of individual predicates.
- This is fine if the number of facts is not very large. But suppose if we want to express simple facts such as greater – than and less – than relationships:
- Consider the following predicates,
greater-than(1,0) greater-than(2,1) greater-than(3,2) less-than(0,1) less-than(1,2) less-than(2,3)
- It is often useful to have computable functions as well as computable predicates.

Example: $gt(2 + 3, 1)$, the value of the plus function is computed given the arguments 2 and 3, and then send the arguments 5 and 1 to gt .

- Predicates: A predicate is a function that returns a Boolean value (true or false). Predicates are used to express properties of objects or relationships between them, and they are critical in logical reasoning, decision-making, and querying data.

1. All pompeians died when the volcano erupted in 79 A.D.

Solution: $\text{erupted}(\text{volcano}, 79) \wedge \forall x: [\text{pompeian}(x) \rightarrow \text{died}(x, 79)]$

2. No mortal lives longer than 150 years.

Solution: $\forall x: \forall t1: \forall t2: \text{mortal}(x) \wedge \text{born}(x, t1) \wedge \text{gt}(t2 - t1, 150) \rightarrow \text{dead}(x, t2)$

t1 and t2 are variable to indicate the time of born and dead. t2-t1 because it will helps us to indicate the total lived years.

3. It is now 1991.

Solution: $\text{now} = 1991$.

4. Alive means not dead.

Solution: $\forall x : \forall t: [\text{alive}(x, t) \rightarrow \sim \text{dead}(x, t)] \wedge [\sim \text{dead}(x, t) \rightarrow \text{alive}(x, t)]$

t represent the time that where he is alive or where he is not dead.

Procedural and Declarative Knowledge

- We can express the knowledge in various forms to the inference engine in the computer system to solve the problems. There are two important representations of knowledge namely, procedural knowledge and declarative knowledge.

Procedural	Declarative
Procedural knowledge provides the knowledge of how a particular task can be accomplished.	Declarative knowledge provides the basic knowledge about something.
Procedural knowledge is also termed as imperative knowledge.	Declarative knowledge is also termed as functional knowledge.
Procedural knowledge revolves around the "How" of the concept.	Declarative knowledge revolves around the "What" of the concept.
Procedural knowledge is process-oriented.	Declarative knowledge is data-oriented.
Procedural knowledge is obtained from actions, experiences, subjective insights, etc.	Declarative knowledge is obtained from principles, procedures, concepts, processes, etc.
Procedural knowledge is represented by a set of rules.	Declarative knowledge is represented by production systems.

Applications:

- Procedural Knowledge is useful when a specific sequence of steps is known and needs to be executed reliably, such as robotic control, decision-making in games, or any task requiring a clear procedure.
- Declarative Knowledge is best suited for systems that need to reason about information, make inferences, or answer queries, such as expert systems, semantic networks, and AI applications in natural language understanding.

Forward vs. Backward Reasoning

- **Forward reasoning**, also known as forward chaining, is a data-driven reasoning approach that starts with known facts and applies inference rules to derive new facts until a goal or conclusion is reached. It operates in a "bottom-up" manner, working from initial data toward a goal.
- **Backward reasoning**, or backward chaining, is a goal-driven reasoning approach that starts with a hypothesis or goal and works backward to see if known facts or data can support the goal. It operates in a "top-down" manner, starting from the goal and working backward.

- Example of Forward Reasoning:
- Scenario: An AI system for animal classification.
- Facts:
 - Fact 1: "The animal has feathers."
 - Fact 2: "The animal can fly."
- Rules:
 - Rule 1: If an animal has feathers, then it is a bird.
 - Rule 2: If an animal is a bird and can fly, then it is likely a sparrow.
- Forward Reasoning Process:
 1. Step 1: Start with the facts: "The animal has feathers" and "The animal can fly."
 2. Step 2: Apply Rule 1: Since the animal has feathers, infer that it is a bird.
 3. Step 3: Apply Rule 2: Since the animal is a bird and can fly, infer that it is likely a sparrow.

- Example of Backward Reasoning:
- Scenario: An AI system determining whether an animal is a sparrow.
- Goal: Determine if the animal is a sparrow.
- Rules:
 - Rule 1: If an animal is a bird and can fly, then it is likely a sparrow.
 - Rule 2: If an animal has feathers, then it is a bird.
- Backward Reasoning Process:
 1. Step 1: Start with the goal: "Is the animal a sparrow?"
 2. Step 2: Find Rule 1, which states that if the animal is a bird and can fly, then it is likely a sparrow. Check if the animal is a bird and can fly.
 3. Step 3: To confirm it is a bird, use Rule 2: If the animal has feathers, then it is a bird.
 4. Step 4: Verify the initial facts: "The animal has feathers" and "The animal can fly." Since both are true, conclude that the animal is likely a sparrow.

Semantic Network

- Semantic networks in AI are graphical structures designed to represent and organize knowledge, enabling machines to understand and process information in human-readable form.
- These networks consist of nodes representing concepts or objects and links denoting relationships between them.
- This structured format provides efficient data retrieval and reasoning.
- Semantic networks are significant in natural language processing, knowledge representation, and information retrieval systems. They focus on capturing the context and meaning of words.
- Lexical Components:
 - Nodes: These represent objects or concepts within the network.
 - Links: They denote the relationships connecting nodes.
 - Labels: Labels attached to nodes and links specify particular objects and relations, providing context.
- Structural Component: In this aspect, nodes and links combine to form a directed graph. Labels are placed strategically on both nodes and links to establish their roles within the network's structure.
- Semantic Component: This component gives meaning to the links and labels associated with nodes. The interpretations of these meanings guide the network's functionality, enabling knowledge representation and inference.
- Procedural Part: Constructors permit the creation of new nodes and links, expanding the network. Destructors, on the other hand, facilitate the removal of nodes and links, ensuring the network remains adaptable and up-to-date.

Applications

- **Natural Language Processing (NLP):** Semantic networks are used to understand and process natural language by modeling word meanings and relationships, enabling tasks like word sense disambiguation, information extraction, and question-answering systems.
- **Expert Systems:** Semantic networks are used in expert systems to represent domain-specific knowledge, allowing systems to reason about data in fields like medicine, engineering, and finance.
- **Knowledge Representation and Reasoning:** They are used to represent complex knowledge structures, such as ontologies in the Semantic Web, which define and interlink terms in a domain to make data interoperable.
- **Conceptual Search Engines:** They enhance search engines by understanding the relationships between search terms, leading to more relevant results.

Advantages

- **Intuitive Representation:** They provide a natural way to represent knowledge that mirrors human cognitive structures, making them easy to understand and visualize.
- **Flexibility:** New concepts and relationships can be easily added to the network, allowing it to evolve.
- **Reasoning Capabilities:** Support for inferencing enables AI systems to derive new information from existing data.
- **Expressiveness:** Can represent a wide range of relationships, from simple hierarchies to complex associative connections.

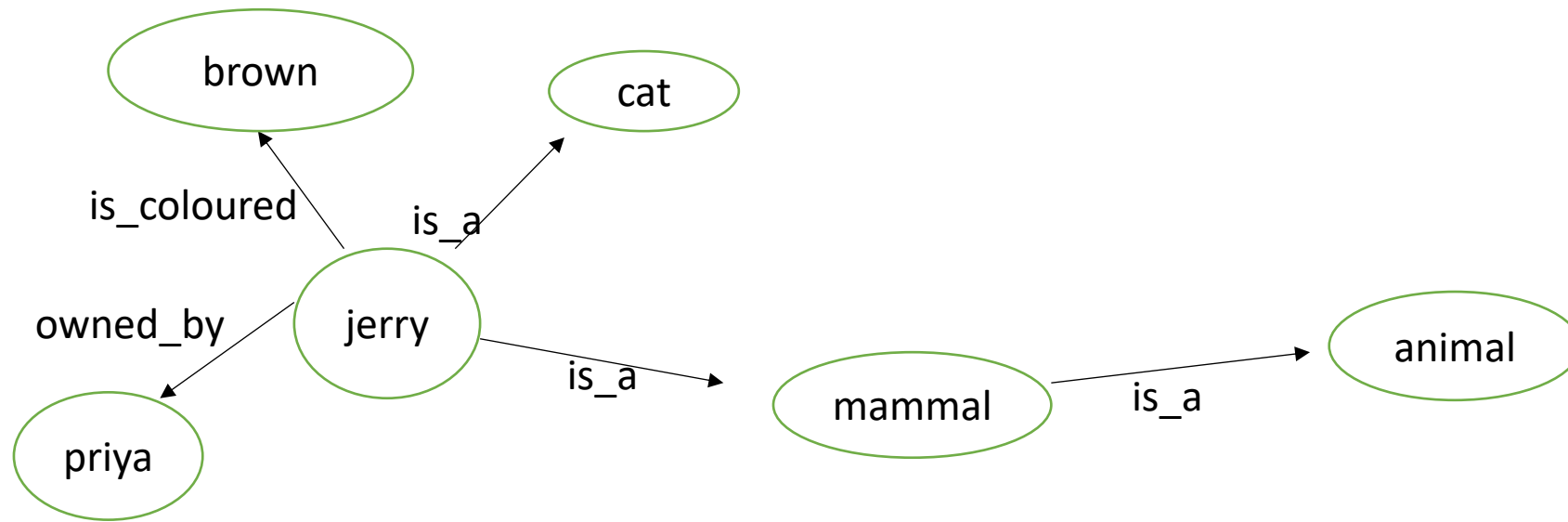
Disadvantages

- **Scalability:** Large semantic networks can become complex and difficult to manage, especially when representing extensive domains of knowledge.
- **Ambiguity and Inconsistency:** Inconsistent or ambiguous relationships can arise, especially when integrating data from different sources.
- **Lack of Standardization:** There is no universal way to represent all types of knowledge, making interoperability challenging across different systems.

Semantic Network

1. Jerry is a cat.
2. Jerry is a mammal
3. Jerry is owned by Priya.
4. Jerry is brown colored.
5. All Mammals are animal.

Semantic Network

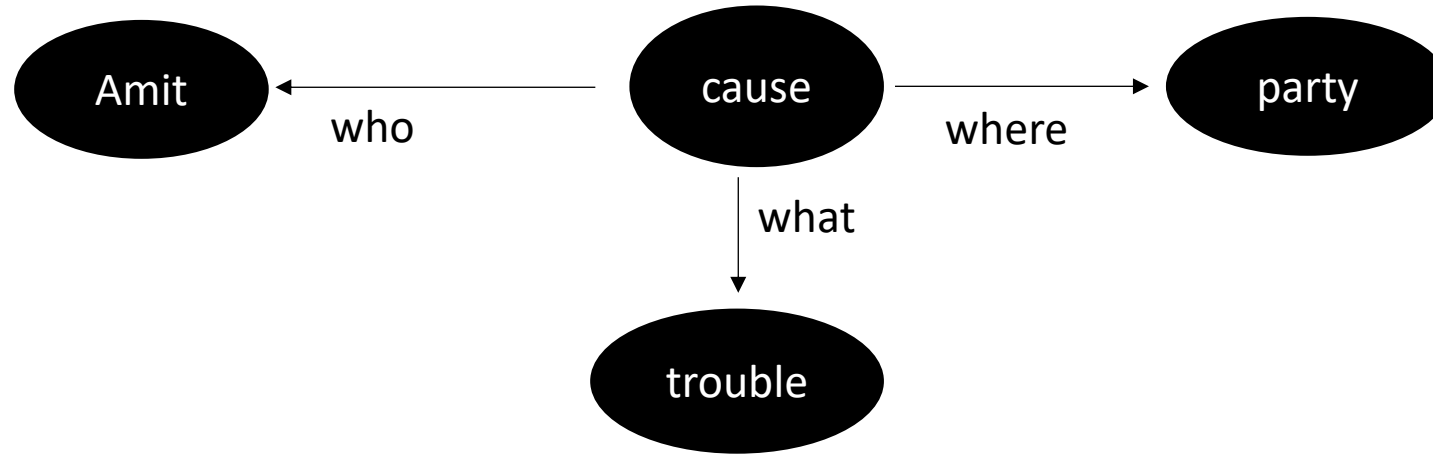


1. John is 7 feet tall.
2. John is taller than Bill.
3. Bill eats peanuts with Shyam.
4. Shyam is a friend of John.

Limitation of Semantic Network

- Binary relations are usually easy to represent, but some times its difficult.

e.g. Amit caused trouble to the party.(convert in ternary)



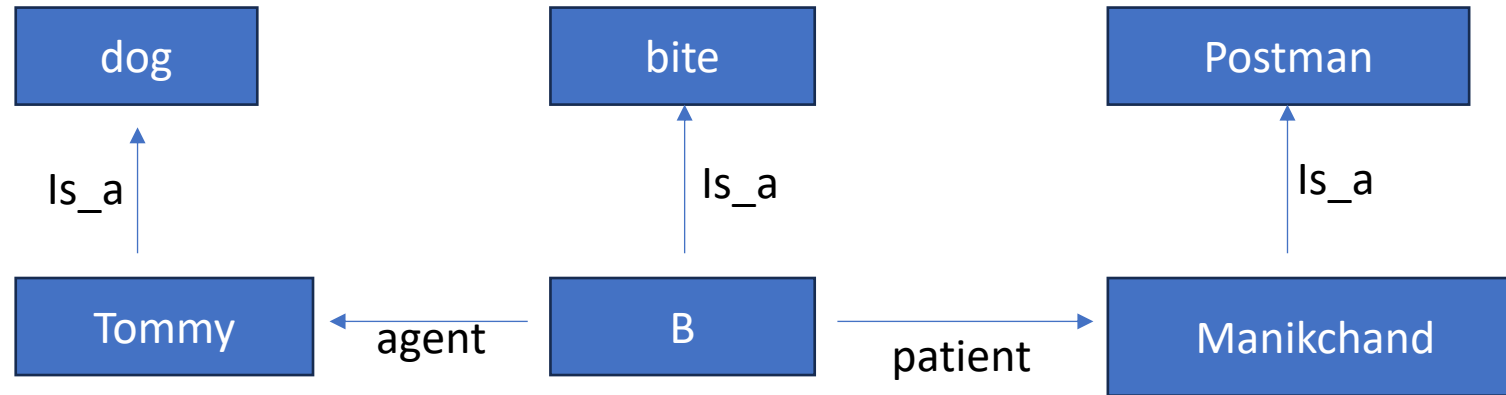
- Other problematic statements...
negation: Amit does not go fishing.
disjunction: Amit eats pizza or fish and chips.
- Quantified statements are very hard for semantic nets.
e.g. Every dog has bitten the postman.

Partitioned Semantic Network.

- Hendrix developed the partitioned semantic network to represent the difference between the description of an individual object or process and the description of a set of objects.
- The set of description involves quantification.
- Hendrix partitioned a semantic network whereby a semantic network, can be divided into one or more networks for the description of an individual.
- The central idea of partitioning is to allow groups, nodes and arcs to be bundled together into units called spaces –fundamental entities in partitioned networks, on same level as nodes and arcs.
- Every nodes and every arc of a network belongs to one or more spaces.

1. Tommy the dog bite Manikchand the postman.

Sol:



1. Every dog has bitten a postman.
2. Every dog has bitten every postman.
3. Every dog in town has bitten the postman.

1. Kittu believes that pizza is tasty.
2. Every student loves to party.

Conceptual dependency

- Conceptual dependency is a theory of natural language processing which mainly deals with representation of semantics of a language(meaning of sentences).
- CD theory was developed by Schank in 1973 to 1975 to represent the meaning of natural language sentences
 - It helps in drawing inferences.(new conclusion)
 - It is independent of the language.
- CD representation of a sentences is not built using the words in the sentences rather built using conceptual primitives which gives the intended meaning of words.

The reason for development of CD:

- To construct computer programs which can understand natural language.
- To make inferences from the statements and also to identify conditions in which two sentences can have similar meaning.
- To provide a means of representation which are language independent.
- To provide facilities for the system to take part in dialogues and answer questions.

- Natural Language Understanding (NLU):** CD helps AI systems understand the meaning of sentences by focusing on conceptual relationships rather than specific syntax, which can be useful in tasks like machine translation or question answering.
- Story Understanding:** CD has been used in AI systems that attempt to "understand" stories by representing events, actions, and their relationships in a clear conceptual structure.
- Inference in Expert Systems:** CD facilitates inferencing by allowing systems to reason about the consequences of actions, the relationships between objects, and the mental states of agents.
- Robotic Systems:** In robotics, CD can be used to represent and reason about the actions of robots and humans, helping systems interact meaningfully with the world based on a conceptual understanding of actions and goals.

Conceptual dependency primitives

- Offers a set of four equal and independent primitive concept.
- From the primitives the word of meaning is built.
 - ACT: Real world actions
 - PP: Real world objects(picture producers)
 - PA: Attributes of objects
 - AA: Attributes of Actions
 - T: Times
 - LOC: locations

Primitives which we have to use in sentences:

- **ATRANS**: Transfer of abstract relationships (e.g., giving, selling, or lending).
- **PTRANS**: Physical transfer or movement (e.g., walking, running).
- **PROPEL**: Applying physical force to move an object.
- **INGEST**: Taking something into the body (e.g., eating, drinking).
- **EXPEL**: Removing something from the body.
- **MTRANS**: Transfer of mental information (e.g., telling, thinking).
- **MBUILD**: Mental construction of new information (e.g., forming conclusions, planning).
- **SPEAK**: Produce sound(e.g., say)
- **MOVE**: Movement of body part by its owner.(e.g., kick)

O: object, P: past tense, I: instrument, D: distance

1. Ram ate apple.
2. Merry went to college yesterday.
3. Susan went to restaurant for dinner.
4. Susan ordered Manchurian noodles.
5. Susan ate Manchurian noodles with a fork.
6. Bill punched John with a broken nose.